Mikkel, Hannibal og Abdallah - Hand-in Assigment 3

Chit Chat service

Mikkel Vincent Hedegaard (mivh@itu.dk)
Hannibal Marcellus Munk (hmun@itu.dk)
Abdallah Jawher Alhamedi (aalh@itu.dk)

October 25, 2026 • Group : Mikkel, Hannibal og Abdallah • https://github.com/Honningbolden/mandatory-exercise-3

---

1 A) - discuss, whether you are going to use server-side streaming, client-side streaming, or bidirectional streaming?

We use server-side streaming to deliver messages to the clients. When a client joins, it opens a stream to the server, which stays open for as long as the client is connected. Through this stream, the client receives a continuous flow of broadcast made up of chat messages, join and leave notifications, each with its Lamport timestamp. This way, users can have full conversations on Chit Chat and are notified when other users join or leave. Sending a message works differently: a client publishes with a unary RPC, since this is an occasional, single request that does not need a persistent stream.

We chose server-side streaming because all messages pass through the server before reaching the other clients, so most of the traffic flows from the server to the clients. This is exactly the pattern server-side streaming is designed for. Client-side streaming would not work, since clients need to receive messages pushed by the server at any time. Bidirectional streaming would also work, but it adds complexity on both the client and the server without much benefit, because publishing is infrequent and fits naturally as a unary call.

2 B) - describe your system architecture - do you have a server-client architecture, peer-to-peer, or something else?

We have a server-client architecture, where a single server acts as the central hub and multiple clients connect to it. Clients do not communicate with each other directly; every interaction goes through the server. A client can join the chat, publish messages, and leave, and the server delivers all broadcasts to the connected clients.

The server keeps track of all currently active clients and their streams. The server keeps track of all clients through local channels stored in a map. When a client publishes a message, or when a client joins or leaves, the server updates its Lamport clock and broadcasts the message to every active client by sending it on each client's stream. Each client connection is served by its own goroutine, so a slow or disconnected client does not block delivery to the others. The server and each client run as separate processes and communicate only through gRPC.

3 C) describe what RPC methods are implemented, of what type, and what messages types are used for communication

We have implemented three message types: User, Message, and Nothing. User contains the username and a Lamport timestamp. Message has a variable sender as a string (username of sender), the content of the message being sent, and the Lamport timestamp. All updates to clients happen in the form of a Message. Nothing is a placeholder for an empty object. We have implemented three RPC methods.

1. The first one is PostMessage, which is unary. The client sends a `Message` to the server and receives `Nothing` once the chat message has been posted. The server then broadcasts the message to all connected clients.
2. The second one is Connect, which is server-side streaming. The client sends its user information as a `User` and, once connected, receives a stream of `Message`. The server creates a channel for the client and runs a loop that uses `select` to wait for messages on that channel, sending each one to the client through the stream. If the users channel is closed or the clients stream stops, the server removes the client from its list and the loop stops.
3. The third one is Disconnect, which is also unary. The client sends its user information as a `User` and receives `Nothing` once it has been disconnected. The server removes the client from its map and closes its channel, which ends the client's Connect stream. Our client does not currently use it, since closing the stream has the same effect, but it is available as an explicit way to leave.

4 D) - describe how you have implemented the calculation of the timestamps

Every Message and User also carries a Lamport timestamp. The server and each client keep their own internal integer clock, starting at 0. 1. Before a client sends a request (Connect or PostMessage) it increments its internal clock and attaches the value. 2. When the server receives a request it updates its internal clock as: clock = max(clock, received) + 1. Both the server and the client have a tick() helper function for this. A client leaving closes its stream. The server detects that and increments its own clock as a local event, since the client doesn't send a timestamp when cancelling the terminal session. 3. The server includes the updated value of its internal clock for each broadcast message, so all clients receive the same timestamp (logical time). When a client receives a broadcast it updates its own clock as: clock = max(clock, received) + 1.

5 D) - provide a diagram, that traces a sequence of RPC calls together with the Lamport timestamps

The diagram below describes the first few messages sent between Alice, Bob and Mikkel. The logs are in the assignment folder. C is the internal clock for each thread.

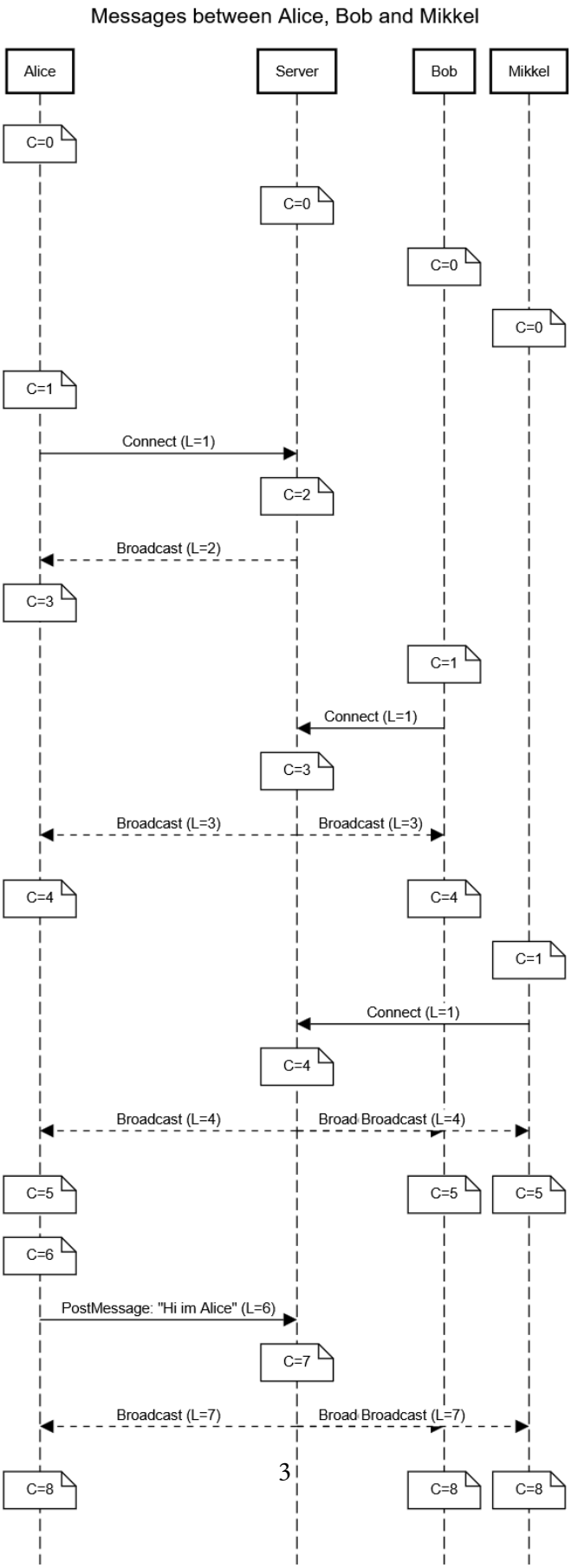


Figure 1: Conversation logs